

# shipcrew

Une équipe d'agents IA qui construit et maintient une application sur GitHub, avec des humains qui pilotent depuis un seul écran.

On décrit le besoin (un PRD). shipcrew le découpe en tâches, les confie à des agents spécialisés qui travaillent en parallèle, fait passer chaque livraison par la CI et une revue de code, puis merge. L'humain n'intervient que lorsqu'une décision lui revient.

**16 features**  
livrées en ~67 min sur une  
démo réelle (11 agents)

**3 agents en parallèle**  
démarrés en ~2 s chacun,  
chacun dans sa copie du  
code

**117 cas testés**  
de garde-fous vérifiés  
automatiquement, pour  
chaque rôle

## Un seul écran : le board

**Board**  
/tmp/claude-1000/-home-juggernaut-Work-Projects-shipcrew/be758984-d0be-4eb1-92c7-21e39a7adddc/scratchpad/e2e/repo  
Imported 2 tasks

E2E math

+ New mission

Plan from PRD

Sync GitHub

Blocked 0

**Backlog** 0 Not scheduled  
+ New task

**Ready** 0 Starts when its ga...

**Running** 0 An agent is wor...

**Review** 0 PR open, CI and ...

**Intervention** 0 Needs a h...

**Merged** 2 Landed on main  

**sumAll** ...  
developer  
Approved  
CI fix 1/3 #3  
#1  
shipcrew/ce64fcc7-...  
\$0.70

**average** ...  
developer  
Needs approval  
Approved CI  
#4 #2  
shipcrew/394d7fad-...  
1 \$0.81

Chaque carte = une tâche = une issue + une PR. Les colonnes disent où elle en est.

# Le parcours d'une tâche

- 1 Le planner transforme le PRD en cartes, avec leurs dépendances et les fichiers que chacune touche.
- 2 Une carte démarre seule quand ses dépendances sont mergées, qu'il reste de la place et qu'aucune autre tâche ne touche les mêmes fichiers.
- 3 Un agent developer code dans sa propre branche, tests compris.
- 4 shipcrew pousse, ouvre la PR, lance la CI et corrige jusqu'à 3 fois si elle échoue.
- 5 Un agent reviewer relit ; si c'est bon, merge automatique ; si c'est sensible, un humain approuve.

## 1 Des agents avec un rôle et des compétences

Chaque rôle est un agent à part, avec ses instructions, ses skills et ses règles. Les règles communes sont écrites une seule fois.

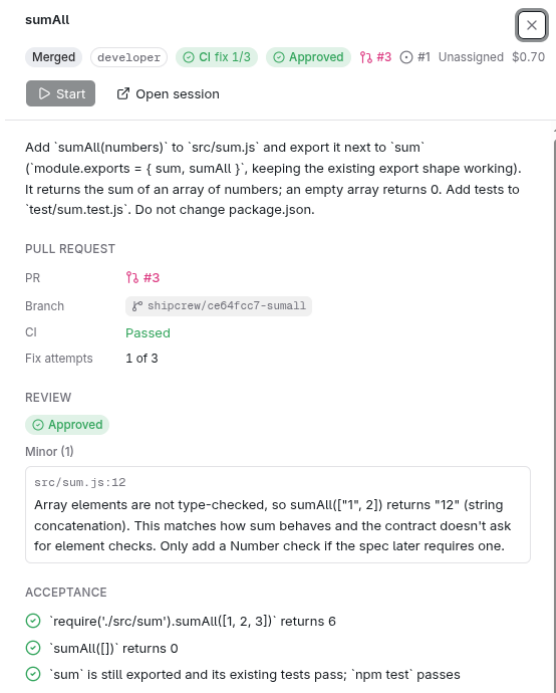
| Rôle                              | Ce qu'il fait                           | Skills                         |
|-----------------------------------|-----------------------------------------|--------------------------------|
| planner                           | PRD en tâches, dépendances, fichiers    | grilling, écriture de PRD      |
| developer                         | code + tests d'une tâche                | TDD, vérification, design-lock |
| reviewer                          | relit la PR, lecture seule              | code-review                    |
| integrator                        | résout les conflits de merge            | merge, vérification            |
| qa                                | teste l'app dans un navigateur          | chrome-devtools, impeccable    |
| security                          | cherche les failles                     | security-review                |
| designer ·<br>scaffolder · devops | style, squelette du projet, déploiement | design-lock, shadcn, vercel    |

Chaque rôle a une liste de commandes autorisées ; tout le reste demande l'accord d'un humain.

## 2 CI/CD branchée sur GitHub (gh)

- Tout passe par le CLI gh : issues, branches, PR, statut de la CI, merge.
- Chaque projet reçoit un workflow GitHub Actions prêt à l'emploi : lint, typage, tests, build, tests navigateur, audit des dépendances.
- CI rouge : les logs d'erreur sont renvoyés à l'agent, qui corrige (3 essais max, puis un humain est prévenu).
- Les merges passent un par un, jamais deux à la fois : main reste toujours verte.
- Les agents ne peuvent pas pousser eux-mêmes : seul shipcrew pousse, et seulement ses propres branches.

## 3 Revue de code systématique



Après une CI verte, un reviewer relit la PR avec les critères d'acceptation de la tâche.

Ses remarques sont classées (bloquant, majeur, mineur) avec le fichier et la ligne, directement sur la carte.

S'il demande des changements, le developer corrige ; au bout de 3 allers-retours, un humain tranche.

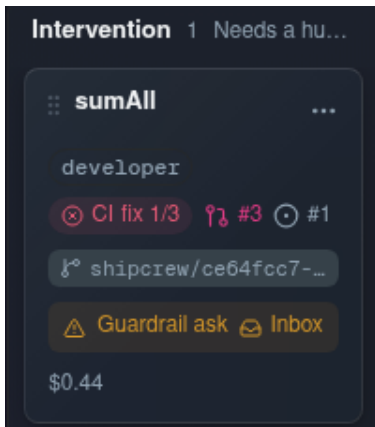
Les zones sensibles (auth, CI, migrations, secrets...) exigent toujours une approbation humaine.

Construit sur omnignt (open source, Apache-2.0), qui pilote les agents Claude Code. Inspiré d'OpenHands, open-swe et shep.

## 4 Qualité testée dans un vrai navigateur

- Chaque tâche arrive avec ses tests de bout en bout (Playwright), qui ouvrent l'application dans Chromium et vérifient les critères d'acceptation.
- Ces tests tournent en local et dans la CI, sur la version de production de l'app.
- L'agent qa parcourt le scénario de démo dans le navigateur, écran par écran, et relève erreurs console et écarts de design.

## 5 Moins de charge mentale pour l'humain



L'humain ne surveille pas des agents : il regarde un board de tâches. Une seule colonne demande son attention : « Intervention ». La carte dit pourquoi (commande à autoriser, merge à approuver, revue bloquée) et mène à l'action en un clic. Glisser une carte suffit à agir : « Ready » la lance, l'assigner à un humain arrête l'agent et lui laisse la main.

## 6 Tout reste observable



En cliquant sur une carte, on voit en direct l'arbre des agents qui travaillent dessus (developer, reviewer, sous-agents).

Un clic ouvre la session : conversation, terminal, diff, fichiers.

Sur chaque carte : statut, branche, PR, CI, verdict de revue et coût.

## Où on en est

- ☒ Board, planner, rôles, garde-fous, boucle PR / CI / revue / merge
- ☒ Testé de bout en bout en local (2 tâches liées, CI corrigée, revue, merge)
- ☐ Test sur un vrai dépôt GitHub (connexion gh à faire)
- ☐ Hébergement sur un serveur pour toute l'équipe, agents isolés dans des conteneurs